

Event-Driven Incident Management System

Event-Driven Incident Management System from scratch, including both a backend incident orchestration engine and a user interface. The platform must automatically detect and manage incidents in a DevOps environment, prioritising infrastructure-level incidents (e.g. server or network failures) while also handling application-level issues (e.g. software errors or slowdowns). Key requirements include: ingestion of simulated alerts from logs, metrics, or monitoring tools; classification of events and routing them into appropriate incident resolution workflows; a workflow engine to manage the incident lifecycle (detection → triage → escalation → resolution → post-mortem); integration with notification channels (email, Slack, SMS, etc.) to alert on-call staff and stakeholders; and operational controls for manual intervention (to acknowledge, escalate, or resolve incidents). A real-time dashboard should provide visibility into active incidents, showing details like severity, ownership (on-call responder), and status. Optional: Teams may include service level agreement tracking or incident analytics (e.g. mean time to resolution, incident frequency by service) for extra insight.

DevOps Scenario

This challenge simulates typical DevOps incident response scenarios, with a focus on infrastructure incidents first. For example, consider a critical server outage or a major network failure: the system should immediately classify it as a high-severity incident, create an incident record, notify the on-call engineer (via SMS, Slack, or similar), and possibly trigger automated remedial actions (such as restarting a service). If the issue isn't resolved promptly, the system should escalate the incident – for instance by notifying a broader support team or a senior engineer – to ensure rapid resolution.

The platform must also handle application-level incidents (which are lower priority). For instance, a spike in error rates or an unresponsive application feature might generate an alert. In these cases, the system could log the incident, notify the relevant application support team (e.g. via email), and track it through resolution without immediately mobilising the highest-tier on-call staff. The difference in handling and urgency between infrastructure and application incidents should be clear in your demo (reflecting their priority order), and your system should show how it routes alerts down different paths based on severity and type.

Technology and Tools

Participants may use any technology stack or libraries to implement the solution, but the core incident management logic and orchestration must be coded by your team. You're free to leverage APIs or services for specific functions (for example, sending emails or SMS, or using a message queue for alerts), and you can choose any programming language and framework for the backend and UI. However, avoid simply configuring an off-the-shelf incident management tool – the objective is to design and build the incident handling workflow yourself. You can optionally employ containerisation, cloud services, or infrastructure-as-code to simulate a realistic deployment, but these are not required given the one-day timeframe.

Deliverables & Presentation

By the end of the hackathon, each team should prepare a short demo and submit the following:

- Incident Management Engine (Backend): A working backend service that ingests incoming alerts, runs the incident workflows, and dispatches notifications. (Include source code in a repository.)
- User Interface (Dashboard): A simple UI (web app or console) that displays active incidents in real time, shows their details (severity, status, assignee), and allows manual actions like acknowledge, escalate, or resolve.
- Simulated Alert Input: A script or tool to generate sample alert events (covering both infrastructure and application incident examples) to test and demonstrate your system's response.
- Presentation/Demo: A brief presentation showing your system in action. Walk through at least one infrastructure incident and one application incident from start to finish, explaining how your design handles each case and highlighting features like notifications, workflow progression, and the dashboard view. Focus on clarity, and be prepared to answer questions about your design decisions and how the system could scale or be extended.